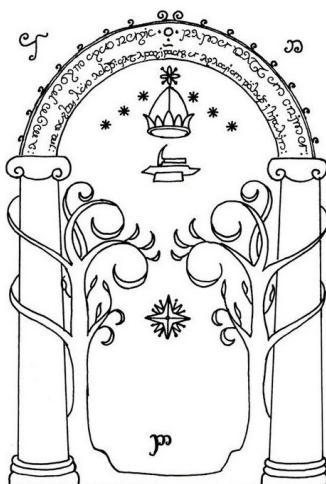




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

4 de septiembre de 2026

Índice

1. Equipo.....	1
2. Repositorio.....	2
3. Dominio.....	2
4. Construcciones.....	2
5. Casos de prueba.....	4
5.1. Casos de aceptación.....	4
5.2. Casos de rechazo.....	4
6. Ejemplos.....	5

1. Equipo

Nombres	Apellidos	Legajo	E-mail
Sofia Belen	Mazzini	65253	smazzini@itba.edu.ar
Alan Gabriel	Navarro	63330	anavarro@itba.edu.ar
Santino	Giambelluca	64697	sgiambelluca@itba.edu.ar
Ignacio	Ferrero Puertolas	64399	iferreropuertolas@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/sgiambelluca/3d-render-TLA>

3. Dominio

El lenguaje a desarrollar es un DSL (*Domain Specific Language*, lenguaje de dominio específico). El dominio seleccionado abarca la descripción declarativa de escenas tridimensionales (cámaras, mallas geométricas, materiales y fuentes de luz) y su compilación hacia un artefacto ejecutable que produce la representación bidimensional (imagen) de dicha escena mediante técnicas de *software rendering*.

El lenguaje utiliza un paradigma declarativo: una escena se describe como un conjunto de entidades (cámaras, luces, materiales y mallas), cada una identificada por un nombre y caracterizada por sus atributos. La posición de cada entidad puede expresarse de forma absoluta, mediante coordenadas, o de forma relativa a otras entidades, mediante expresiones aritméticas y transformaciones (traslación, rotación y escalado) que el compilador resuelve íntegramente en tiempo de compilación. De esta manera, quien escribe la escena no necesita calcular a mano las coordenadas finales de cada objeto.

El *backend* del compilador procesa estas definiciones y genera un documento HTML con JavaScript embebido que, al ejecutarse sobre un *runtime* de rasterización propio (implementado una única vez sobre la API de `<canvas>` del navegador), produce la imagen final de la escena. Este enfoque evita cualquier dependencia de aceleración gráfica por *hardware*, resultando en un artefacto de salida autocontenido y ejecutable en cualquier navegador sin instalación adicional.

4. Construcciones

(I). Estructura de la escena: Una escena (*scene*) agrupa, entre llaves (`{ }`), las entidades que la componen: cámaras, luces, materiales, mallas y la configuración de renderizado. Cada entidad se declara indicando su tipo, un identificador, opcionalmente un subtipo y una posición, seguidos de un bloque con sus atributos. Cada atributo se escribe como su nombre seguido de su valor (e.g., `radius 0.7`), sin separadores adicionales.

(II). Cámara (*camera*): Entidad única por escena que define el punto de vista desde el cual se renderiza. Su posición se indica con `at` y sus atributos son el punto de mira (`look_at`), que puede ser un vector o el identificador de otra entidad, y el campo de visión o FoV (*Field of View*, `fov`), que debe ser un ángulo estrictamente mayor a 0° y estrictamente menor a 180° .

(III). Luces (*light*): Fuentes lumínicas de dos subtipos: direccional (*directional*), definida por un vector de dirección (`direction`) no nulo, y puntual (*point*), definida por su posición. Ambas tienen color (`color`) e intensidad (`intensity`), siendo esta última un valor numérico en el rango $[0, 1]$.

(IV). Materiales (*material*): Definen propiedades de apariencia reutilizables por múltiples mallas: el color (`color`), que admite transparencia mediante el canal *alpha*, y el brillo especular (*shininess*), un entero no negativo donde 0 representa un material completamente mate.

(V). Mallas geométricas (mesh): Primitivas geométricas de tres subtipos, cada una con sus atributos dimensionales y una referencia a un material (`material`). Su posición (`at`) indica el centro de la malla:

- **Esfera (sphere):** su radio (`radius`), un número positivo.
- **Bloque (box):** sus dimensiones (`size`), un vector de tres componentes positivas (ancho, alto, profundidad). Esto permite representar prismas de base rectangular; un cubo se obtiene con tres componentes iguales, o bien indicando un único número como abreviatura (*e.g.*, `size 2`).
- **Plano (plane):** los planos son siempre finitos: representan un rectángulo definido por sus dimensiones (`size`), un vector de dos componentes positivas (ancho, profundidad). Por defecto se ubica en posición horizontal, con vector normal (0, 1, 0), y puede orientarse en cualquier dirección mediante una rotación (ver construcción VII).

(VI). Posiciones relativas y expresiones aritméticas: Toda posición, vector o valor numérico puede escribirse como una expresión que combine literales y referencias a atributos de otras entidades: la posición de una entidad (`pelota.pos`), sus atributos (`mesa.size`) y sus componentes (`mesa.size.y`). Se admiten suma y resta entre vectores, producto y división de un vector por un escalar, las cuatro operaciones entre escalares y el uso de paréntesis. Todas las expresiones se evalúan en tiempo de compilación, por lo que el artefacto generado solo contiene las coordenadas finales (ver Cód. (6.3)).

(VII). Transformaciones: Las mallas admiten las operaciones de traslación (`translate`, con un vector de desplazamiento), rotación (`rotate`, con un ángulo por cada eje) y escalado (`scale`, con un factor único o un vector con un factor por eje). Se aplican sobre el centro de la malla, en el orden en el que están escritas, y también se resuelven en tiempo de compilación.

(VIII). Referencias entre entidades: Una entidad puede referenciar a cualquier otra entidad de la misma escena, sin importar si esta fue declarada antes o después en el programa (*e.g.*, una malla referencia un material por su identificador, o una luz se ubica relativa a una malla). Durante el análisis semántico, el compilador construye el grafo de dependencias entre los atributos de las entidades, lo resuelve en orden topológico y rechaza el programa si existe una dependencia cíclica o una referencia a un identificador inexistente (ver Cód. (6.2) y Cód. (6.4)).

(IX). Renderizado (render): Construcción que especifica la resolución de la imagen de salida y el nombre del archivo a generar (*e.g.*, `render (640, 480) to "esfera.html"`).

(X). Tipos de datos: Cadenas de texto ("`...`"), numéricos (enteros y de punto flotante), ángulos con sufijo (`60deg`), colores hexadecimales con canal *alpha* opcional (`#rrggbb` o `#rrggbbaa`, donde `00` es completamente transparente y `ff` completamente opaco), vectores de dos y tres componentes (`(x, y)` y `(x, y, z)`), identificadores y referencias a atributos de otras entidades (`id.atributo`).

(XI). Comentarios: De línea simple utilizando `//`.

(*) Nota sobre nomenclatura: la posición de una entidad se declara con la palabra reservada `at` y, al ser referenciada desde otra entidad, se accede mediante `.pos` (abreviatura de *position*).

5. Casos de prueba

5.1. Casos de aceptación

Se proponen los siguientes casos iniciales de prueba de aceptación:

- (I). Un programa que defina una escena mínima, compuesta únicamente por una cámara y la configuración de renderizado, sin ninguna malla ni luz.
- (II). Un programa que defina una escena completa: una cámara, una luz direccional, un material y un bloque que utilice dicho material.
- (III). Un programa en el que dos o más mallas distintas utilicen el mismo material.
- (IV). Un programa que defina una luz puntual ubicada mediante su posición.
- (V). Un programa que declare múltiples luces dentro de la misma escena.
- (VI). Un programa que defina una esfera a partir de su radio.
- (VII). Un programa que defina un plano finito de ancho y profundidad dados, representando el piso de la escena.
- (VIII). Un programa que defina un bloque con dimensiones distintas en cada eje (un prisma de base rectangular) y otro con una única dimensión (un cubo).
- (IX). Un programa que declare una cámara con un campo de visión cercano al límite superior permitido (179°).
- (X). Un programa que declare un material con brillo especular nulo (material completamente mate).
- (XI). Un programa que configure una resolución de salida rectangular no cuadrada (e.g., 1920×1080).
- (XII). Un programa en el que una entidad referencie a otra declarada más adelante en el programa (e.g., una malla que utiliza un material definido después de ella), como en Cód. (6.2).
- (XIII). Un programa en el que la posición de una entidad se calcule a partir de la posición y las dimensiones de otra (e.g., una esfera apoyada sobre un bloque), como en Cód. (6.3).
- (XIV). Un programa que aplique traslación, rotación y escalado sobre una malla.
- (XV). Un programa que declare un material con un color semitransparente.

5.2. Casos de rechazo

Se proponen los siguientes casos iniciales de prueba de rechazo:

- (I). Un programa en el que una malla utilice un material que no fue declarado en ninguna parte de la escena.
- (II). Un programa en el que dos entidades de la misma escena compartan el mismo identificador.

- (III). Un programa en el que una luz direccional declare como dirección el vector nulo (0, 0, 0), que no puede normalizarse.
- (IV). Un programa en el que una cámara declare un campo de visión fuera del rango físicamente válido (e.g., 200°).
- (V). Un programa en el que una luz declare un valor de intensidad fuera del rango permitido (e.g., -0.5 o 1.5).
- (VI). Un programa con una dependencia cíclica entre entidades (e.g., dos mallas cuyas posiciones se definen cada una en función de la otra), como en Cód. (6.4).
- (VII). Un programa en el que un bloque o un plano declare alguna dimensión nula o negativa.
- (VIII). Un programa con una expresión que opere valores incompatibles (e.g., sumar un vector de dos componentes con uno de tres, o sumar un color a una posición).
- (IX). Un programa en el que un color declare un canal *alpha* incompleto (e.g., #2e86d o #2e86de8).

6. Ejemplos

La escena más simple posible aceptada por el lenguaje: solo una cámara y la configuración de renderizado, sin ninguna malla ni luz (Cód. (6.1)).

```
scene "vacía" {  
  camera main at (0, 2, 5) {  
    look_at (0, 0, 0)  
    fov 60deg  
  }  
  
  render (800, 600) to "frame.html"  
}
```

Código 6.1: Escena mínima, compuesta únicamente por una cámara y la configuración de renderizado.

En Cód. (6.2), una cámara observa una esfera con material propio, iluminada por una luz puntual ubicada 3.5 unidades por encima de la esfera. Tanto la esfera como el material se declaran después de las entidades que los referencian, lo cual es válido porque el orden de declaración no importa.

```
scene "esfera_lampara" {
  camera lateral at (3, 2, 4) {
    look_at pelota
    fov 50deg
  }

  light lampara : point at pelota.pos + (0, 3.5, 0) {
    color #88aaff
    intensity 0.8
  }

  mesh pelota : sphere at (0, 0.5, 0) {
    radius 0.7
    material azul
  }

  material azul {
    color #2e86de
    shininess 64
  }

  render (640, 480) to "esfera.html"
}
```

Código 6.2: Escena con una esfera iluminada por una luz puntual, con referencias a entidades declaradas más adelante.

En Cód. (6.3), la posición de cada objeto se calcula a partir de otros: la mesa se apoya sobre el piso, el jarrón sobre la mesa y el cubo de vidrio, semitransparente, se ubica al costado de la mesa, rotado y escalado. Ninguna coordenada final debe calcularse a mano.

```
scene "mesa_con_jarron" {
  camera frente at (0, 3, 6) {
    look_at mesa
    fov 45deg
  }

  light sol : directional {
    direction (-1, -2, -1)
    color #ffffff
    intensity 0.9
  }

  material baldosa { color #d0d0d0    shininess 0 }
  material madera  { color #8b5a2b    shininess 8 }
  material vidrio  { color #aee4ff80  shininess 128 } // alpha 50 %

  mesh piso : plane at (0, 0, 0) {
    size (10, 10)
    material baldosa
  }

  mesh mesa : box at piso.pos + (0, 0.5, 0) {
    size (2, 1, 1)
    material madera
  }

  // el jarrón queda apoyado sobre la mesa
  mesh jarron : sphere
    at mesa.pos + (0, mesa.size.y / 2 + jarron.radius, 0) {
      radius 0.25
      material vidrio
    }

  // el cubo queda al costado de la mesa, rotado y escalado
  mesh cubo : box at mesa.pos + (mesa.size.x / 2 + 1, 0, 0) {
    size 0.5
    material vidrio
    rotate (0, 45deg, 0)
    scale 1.5
    translate (0, -0.125, 0)
  }

  render (1920, 1080) to "mesa.html"
}
```

Código 6.3: Escena con posiciones relativas, transformaciones y un material semitransparente.

Por último, Cód. (6.4) muestra un programa que debe ser rechazado: la posición de cada esfera depende de la posición de la otra, por lo que el compilador detecta una dependencia cíclica.

```
scene "ciclo" {
  camera main at (0, 2, 5) {
    look_at a
    fov 60deg
  }

  material gris { color #888888  shininess 0 }

  mesh a : sphere at b.pos + (1, 0, 0) { // depende de b
    radius 0.5
    material gris
  }

  mesh b : sphere at a.pos - (1, 0, 0) { // depende de a: ciclo
    radius 0.5
    material gris
  }

  render (640, 480) to "ciclo.html"
}
```

Código 6.4: Programa rechazado por una dependencia cíclica entre las posiciones de dos mallas.